



Java Coding Standards

Good Code vs Bad Code Practices
Real-time Examples with code



1. Use Meaningful Variable Names (Basic)

Why?

1. Short names make the code confusing.
2. Meaningful names explain the variable's purpose.
3. Makes the code self-explanatory.

Bad

java

```
int t = 11; // What is 't'?
```

- Unclear and hard to understand.
- No context about its purpose.

Good

java

```
int teamSize = 11;
```

- Clearly indicates it's a cricket team size.
- Easy to read and understand.



2. Avoid NullPointerException 🚫 🤔 (Basic)

Why?

1. NullPointerException happens when something is null.
2. Comparing directly without null checks is risky.
3. Use constants or Optional to handle null safely.

Bad ❌

```
java

if (player.getName().equals("Virat")) {
    // Do something
}
```

- Throws an error if `player.getName()` is null.
- Leads to NullPointerException.

Good ✅

```
java

if ("Virat".equals(player.getName())) {
    // Do something
}
```

- Constant is always safe to compare.
- Avoids null-related issues.



3. Use Constants for Magic Numbers 🧙‍♂️ 1234 (Intermediate)

Why?

1. Using raw numbers makes code unclear.
2. Constants explain the meaning of numbers.
3. Makes the code easy to maintain.

Bad ❌

java

```
if (runs == 183) {  
    // Do something  
}
```

- No idea what 183 represents.
- Hard to maintain and understand.

Good ✅

java

```
private static final int VIRAT_HIGHEST_SCORE = 183;  
if (runs == VIRAT_HIGHEST_SCORE) {  
    // Do something  
}
```

- Constant gives meaning to the number.
- Makes the code readable and clear.



4. Avoid Long Methods 🚀🔧 (Intermediate)

Why?

1. Long methods are hard to read and understand.
2. Debugging and reusing becomes difficult.
3. Split tasks into smaller methods for clarity.

Bad ❌

java

```
public void processTeamData() {  
    // 100+ lines of code  
}
```

- All logic in one place makes it complex.
- Hard to debug or reuse.

Good ✅

java

```
public void processTeamData() {  
    validateTeam();  
    calculateRunRate();  
    updateDatabase();  
}
```

- Breaks logic into smaller methods.
- Easier to read and reuse.



5. Handle Exceptions Properly ⚠️✅ (Advanced)

Why?

1. Printing stack traces doesn't help with debugging.
2. Use meaningful logs to track errors.
3. Rethrow exceptions when necessary.

Bad ❌

java

```
try {  
    // Code  
} catch (Exception e) {  
    e.printStackTrace();  
}
```

- Only prints the error without context.
- Makes debugging harder.

Good ✅

java

```
try {  
    // Code  
} catch (IOException e) {  
    logger.error("Error processing Dhoni's match stats", e);  
}
```

- Logs helpful messages for debugging.
- Provides clear information about the issue.



6. Use Enums for Constants (Advanced)

Why?

1. Hardcoded strings are prone to typos.
2. Enum provides better type safety.
3. Makes the code clean and organized.

Bad

java

```
public static final String PLAYER_ACTIVE = "ACTIVE";
```

- Strings can be misspelled.
- Hard to manage and prone to errors.

Good

java

```
public enum PlayerStatus {  
    ACTIVE, RETIRED;  
}
```

- Reliable and type-safe.
- Easier to maintain and avoid errors.



7. Use Streams Wisely 🌊📊 (Advanced)

Why?

1. Loops for processing collections are verbose.
2. Streams make the code concise and clean.
3. Reduces boilerplate and improves readability.

Bad ❌

java

```
List<String> captains = new ArrayList<>();
for (Player player : players) {
    if (player.isCaptain()) {
        captains.add(player.getName());
    }
}
```

- Repetitive and lengthy logic.
- Hard to read and maintain.

Good ✅

java

```
List<String> captains = players.stream()
    .filter(Player::isCaptain)
    .map(Player::getName)
    .collect(Collectors.toList());
```

- Concise and clear logic.
- Reduces lines of code significantly.





To get more **PDF**
with **simple explanations**

Follow us on Instagram
@java_tech_community

and Visit
JavaTechCommunity.com
for more.